11.6.2 Living Documentation for legacy applications

Many organizations have large legacy applications that are still very much in use and still need regular maintenance, updates, and releases. Many of these applications have few unit or integration tests and low test coverage, and they're often lacking technical or functional documentation.

The lack of automated tests for this sort of application makes it harder and riskier to deliver new features or bug fixes quickly. Releases are delayed by long testing cycles and regression issues. But for many organizations, rewriting the entire application isn't a viable proposition.

BDD offers some possible approaches that can relieve these symptoms and help teams to deliver changes to their legacy applications more safely and efficiently. One popular strategy that many teams adopt is to retrofit the legacy application with high-level acceptance tests, typically web tests for a web application. These acceptance tests both describe and document the existing system and help reduce the risk of regressions when new features are introduced. The tests are written in the same style as the automated acceptance criteria you've seen elsewhere in this book, and often with the same BDD-focused tools (such as Cucumber, SpecFlow, and JBehave). The BDD reporting capabilities of these tools are a great way to document and communicate how people believe the application should behave.

Retrofitting unit testing is traditionally very difficult to do effectively, because unit tests written too long after the code was written tend to be quite superficial. BDD unit tests specify the behavior of classes and components within these applications and are often the only place that this behavior is documented. When these specifications don't exist, it can be difficult to invent them after the fact, as it involves a deep understanding of how each class or component is expected to behave.

Despite these difficulties, BDD unit tests can provide excellent technical documentation, even for legacy applications. Some teams with mission-critical legacy applications that still require frequent and significant changes use BDD tests to document the most critical or high-risk parts of their application first, before expanding into less critical functionality. Unit tests are written in the spirit of documenting the current application behavior and giving code samples for how to use each class. This is a great way to provide technical documentation for the more critical parts of the application and to build up high-quality test coverage.

11.7 Summary

In this chapter you learned what living documentation means in a BDD project:

- Living documentation is a way of automatically reporting both what an application is intended to do and how it does it.
- Living documentation provides information about features and requirements in addition to individual test results.